Introdução à Paradigmas de Projeto de Algoritmos

Estruturas de Dados e Introdução à Projeto e Análise de Algoritmos

Prof. Luis Souza

Introdução à Projeto de Algoritmos

- O projeto de algoritmos requer abordagens adequadas:
  - A forma como um algoritmo **modela** o problema pode levar a um desempenho ineficiente
  - O algoritmo pode não conseguir resolver o problema em tempo viável de acordo com a **modelagem** adotada
- Principais paradigmas:
  - Indução matemática
  - Recursividade
  - Algoritmos de tentativa e erro
  - Divisão e conquista
  - Algoritmos gulosos
  - Programação dinâmica
  - Algoritmos aproximados

Introdução à Projeto de Algoritmos

- Indução matemática e recursividade: já visitados nos módulos anteriores (serão revisitados)
- Algoritmos tentativa-e-erro: analisa-se todas as soluções possíveis do problema sem considerar qualquer critério para evitar a análise de certas soluções baseadas em histórico ou memorização
  - implementa a idéia mais simples possível para solucionar o problema -> abordagem é na maioria das vezes inviável

Introdução à Projeto de Algoritmos

- Algoritmos de divisão e conquista, gulosos e programação dinâmica implementam estratégias que baseiam-se na decomposição de problemas complexos em outros mais simples, cujas soluções são combinadas para fornecer uma solução para o problema original
  - D&C: normalmente implementa soluções recursivas
  - PD e AG: tendem a entregar soluções modeladas de forma iterativa
- Algoritmos aproximados tentam resolver o problema quando a solução é considerada difícil (tempo exponencial)

Introdução à Projeto de Algoritmos

- Não existem paradigmas melhores que outros, mas que modelam de forma mais eficiente alguns problemas
- Exemplo: um paradigma pode resolver um problema com complexidade **O(2^n)** enquanto outro pode resolver o mesmo problema com complexidade **$O(n^3)$**
  - Como ordenar um vetor de inteiros?
  - Como realizar o produto entre n matrizes de modo que o número de operações seja o menor possível?
  - **Problema da mochila:**
    - Considere n itens a serem levados para uma viagem, dentro de uma mochila de capacidade L que não pode comportar todos os itens
    - Cada item tem um peso w_i e uma utilidade c_i . Quais itens escolher de modo que a utilidade total dos itens levados seja a maior possível?

Indução Matemática

- É útil para provar asserções sobre a correção e a eficiência de algoritmos
- Consiste em inferir uma lei geral a partir de instâncias particulares
- Seja T um teorema que tenha como parâmetro um número natural n,
- Para provar que T é válido para todos os valores de n, prova-se que:
  - T é válido para n = 1
  - Para todo n > 1, se T é válido para n − 1, então T é válido para n

Indução Matemática

- A condição 1 é chamada de **passo base**
- Provar a condição 2 é geralmente mais fácil que provar o teorema diretamente (pode-se usar a asserção de que T é válido para n − 1)
  - Esta afirmativa é chamada de **hipótese de indução** ou **passo indutivo**
- As condições 1 e 2 implicam T válido para n = 2, o que junto com a condição 2 implica T também válido para n = 3, e assim por diante

Indução Matemática (exemplo)

- $S(n) = 1 + 2 + \cdots + n = n(n + 1)/2$:
  - Para n = 1 a asserção é verdadeira, pois $S(1) = 1 = 1 \times (1 + 1)/2$ (**passo base**)
  - Assume-se que a soma dos primeiros n números naturais S(n) é $n(n + 1)/2$ (**hipótese de indução**)
  - Pela definição de S(n) sabe-se que $S(n + 1) = S(n) + n + 1$,
  - Usando a hipótese de indução
    - $S(n + 1) = n(n + 1)/2 + n + 1$

      $= (n + 1)(n + 2)/2$, que é exatamente o que se quer provar

Indução Matemática (exemplo)

- A solução de uma equação de recorrência pode ser difícil de ser obtida
  - Pode ser mais fácil tentar “adivinhar” a solução ou obter um limite superior para a ordem de complexidade
- Adivinhar a solução funciona bem quando se está interessado apenas em um limite superior ao invés da solução exata
  - Mostrar que um certo limite existe é mais fácil do que obter o próprio limite
- Ex.: $T(2n) \leq 2T(n) + 2n - 1$, $T(2) = 1$, definida para valores de n que são potências de 2:
  - O objetivo é encontrar um limite superior na notação O, onde o lado direito da desigualdade representa o pior caso

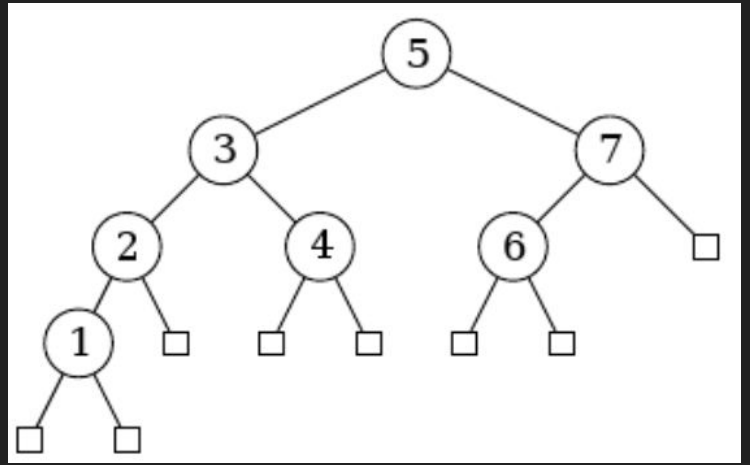
Indução Matemática (exemplo)

- $T(2n) \leq 2T(n) + 2n - 1$, $T(2) = 1$, definida para valores de n que são potências de 2:
  - Procura-se f(n) tal que $T(n) = O(f(n))$, mas fazendo com que f(n) seja o mais próximo possível da solução real para T(n)
  - Hipótese: $f(n) = n^2$
  - Provar que $T(n) = O(f(n))$ utilizando indução matemática em n:
    - Passo base: $T(2) = 1 \leq f(2) = 4$
    - Passo indutivo: provar que $T(n) \leq f(n)$ implica $T(2n) \leq f(2n)$:
      - $T(2n) \leq 2T(n) + 2n - 1$, (def. da recorrência)
      - $\leq 2n^2 + 2n - 1$, (hipótese de indução)
      - $\leq 2n^2$,

      que é exatamente o que se quer provar. Logo, $T(n) = O(n^2)$

Recursividade

- Definição: um procedimento que chama a si mesmo, direta ou indiretamente
- Permite descrever algoritmos de forma mais elegante, clara e concisa, especialmente problemas naturalmente recursivos ou que utilizam estruturas recursivas
- Exemplo: árvores binárias



Recursividade - Propriedades

- Utiliza-se uma pilha para armazenar os dados usados em cada chamada de um procedimento que ainda não terminou (pilha de recursão)
  - Todos os dados não globais vão para a pilha, registrando o estado corrente da computação
  - Quando uma ativação anterior prossegue, os dados da pilha são recuperados
- No caso do caminhamento em ordem:
  - para cada chamada recursiva, o valor de p (nó da chamada) e o endereço de retorno da chamada recursiva são armazenados na pilha
  - Quando p == null -> o procedimento retorna para quem chamou utilizando o endereço de retorno que está no topo da pilha

Recursividade - Propriedades

- Procedimentos recursivos introduzem a possibilidade de iterações que podem não terminar: existe a **necessidade** de considerar um critério de parada (ou problema de terminação ou caso base)
- É fundamental que a chamada recursiva a um procedimento P esteja sujeita a uma condição B, a qual se torna não-satisfeita em algum momento da computação (o processo indutivo deve convergir para um caso basal de computação)
- Esquema para procedimentos recursivos: composição C de comandos S_i e P: P ≡ if B then C[S_i , P]
- Define-se uma função f(x) para demonstrar a terminação da execução do procedimento, sendo x o conjunto de variáveis do programa, tal que:
  - f(x) ≤ 0 implica no critério de parada
  - f(x) é decrementada a cada iteração (convergir para parada)

Recursividade - Propriedades

- De forma simplificada, a terminação é garantida pela associação de um parâmetro *n* para P (no caso por valor) decrementado a cada chamada recursiva
  - P ≡ if *n* > 0 then C[S_i , P(n − 1)]
- É necessário garantir que o nível mais profundo da recursão é finito, e também possa ser mantido pequeno -> cada chamada recursiva usa uma parcela de memória para acomodar variáveis e endereços de retorno

Recursividade - Quando não usar

- Nem todo problema de natureza recursiva deve ser resolvido com um algoritmo recursivo
  - Estes casos podem ser caracterizados pelo esquema P ≡ if B then (S, P) (podem ser definidos recursivamente)
  - São facilmente transformáveis em uma versão não recursiva P ≡ (x := x0 ; while B do S)
- Algum exemplo claro de não-uso de modelagem recursiva???

Fibonacci

Cálculo dos números de Fibonacci:

- $f_0 = 0$, $f_1 = 1$,
- $f_n = f_{n-1} + f_{n-2}$ para $n \geq 2$.

Solução: $fn = \frac{1}{\sqrt{5}}[\Phi^n - (-\Phi)^{-n}]$, onde $\Phi = \frac{1+\sqrt{5}}{2} \approx 1,618$ é a razão de ouro.

```
def fibonacci(n):
    if (n<2)
        return n
    else
        return fibonacci(n-1) + fibonacci(n-2)
```

Fibonacci

- O cálculo de fibonacci por modelagem recursiva é extremamente ineficiente
  - recalcula o mesmo valor várias vezes
  - tem complexidade O(Φ^n) para o cálculo de f(n)
  - Considerando que a medida de complexidade de tempo f(n) é o número de adições, e cada adição tem custo de O(1), então f(n) = O(Φ^n)

Fibonacci (versão iterativa)

```
def fibonacciIter(n):
    i = 1
    k = F = 0
    for K in range(n+1):
        F+=i
        i=F-i
    return F
```

- Tem complexidade O(n) (pode ser melhorado?)
- Deve-se evitar uso de recursividade quando existe solução óbvia por iteração

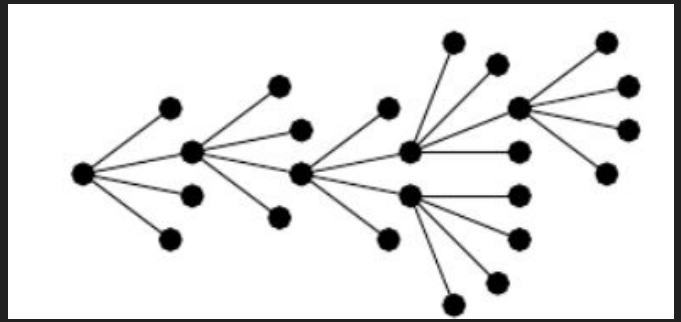
8GB de RAM

Comparação versões recursiva e iterativa:

n	10	20	30	50	100
Recursiva	8 ms	1 s	2 min	21 dias	10^9 anos
Iterativa	1/6 ms	1/3 ms	1/2 ms	3/4 ms	1,5 ms

Tentativa e Erro

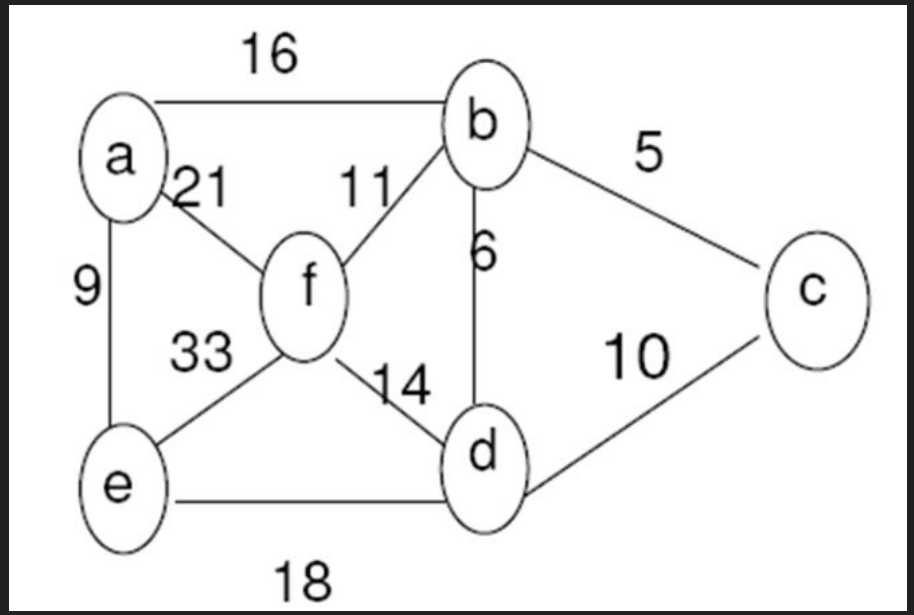
- Algoritmos de tentativa e erro exploram **exaustivamente** todas as soluções possíveis de um problema com a finalidade de obter a solução desejada
- As soluções são testadas indiscriminadamente -> não utiliza critérios para eliminar outras soluções que não poderão ser melhores que a obtida no estágio considerado
  - As soluções são enumeradas de modo similar ao percurso em uma árvore que descreve todas as soluções
  - É comum a “árvore” de soluções crescer exponencialmente (PROBLEMA!!!)



Tentativa e Erro

- Algoritmos de tentativa e erro não seguem regra fixa de execução
  - Passos em direção à solução final são tentados e registrados
  - Caso passos tomados sejam espúrios eles podem ser retirados e apagados do registro (lembra do Sudoku?)
- Quando a pesquisa na árvore de soluções cresce rapidamente é necessário empregar **algoritmos aproximados** ou **heurísticas** que não garantem a solução ótima global mas agilizam a busca por soluções ótimas locais (e talvez globais)

Tentativa e Erro - Exemplo



- Qual o caminho de custo mínimo de *a* até *c*?
- **TODOS** os caminhos são enumerados:
  - a ⇒ b ⇒ c: 21
  - a ⇒ b ⇒ d ⇒ c: 32
  - a ⇒ b ⇒ f ⇒ d ⇒ c: 51
  - …

Divisão e Conquista

- O paradigma de divisão-e-conquista consiste em:
  - Dividir o problema a ser resolvido em subproblemas menores e independentes
  - Encontrar soluções para as partes
  - Combinar as soluções obtidas em uma solução global
- Utiliza-se recursão para dividir e combinar:
  - Dada uma entrada, se ela é suficientemente simples, obtém-se diretamente uma saída correspondente
  - Caso contrário, ela é decomposta em entradas mais simples, para as quais se aplica o mesmo processo, obtendo saídas correspondentes que são então combinadas em uma saída para a entrada original

Divisão e Conquista

- Exemplo: encontrar o maior e o menor elemento de um vetor de inteiros, A[1..n], n ≥ 1:

```
def minmax(lista):

    if(len(lista) == 0):
        raise ValueError('Cannot find the maximum of an empty list.')

    if len(lista) == 1:
        return (lista[0], lista[0])

    _min, _max = minmax(lista[1:])

    if lista[0] < _min:
        _min = lista[0]

    if lista[0] > _max:
        _max = lista[0]

    return (_min, _max)
```

Divisão e Conquista

- Lado a lado, obtém-se:

$$
\begin{aligned}
f(n) &= 2^{i-1} f(n/2^{i-1}) + \sum_{k=1}^{i-1} 2^k \\
&= 2^{i-1} f(2) + 2^i - 2 \\
&= 2^{i-1} + 2^i - 2 \\
&= \frac{3n}{2} - 2
\end{aligned}
$$

- Logo, f(n) = 3n/2 − 2 para o melhor caso, pior caso e caso médio (solução ótima)
- Outros exemplos de **D&C**: Busca Binária, Merge Sort, Quick Sort, Fibonacci recursivo.

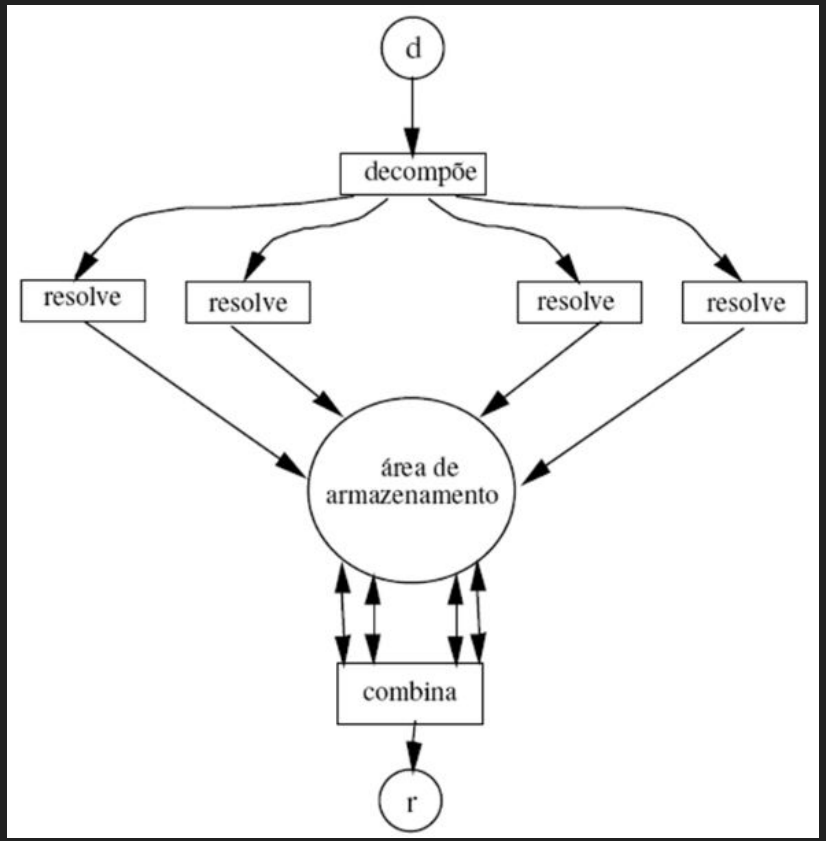
Programação Dinâmica

- A programação dinâmica visa
  - dividir o problema original em subproblemas mais simples e resolvê-los,
  - armazenar os resultados em uma tabela para reuso
  - modelagem usualmente iterativa (bottom-up)
- A ideia básica da programação dinâmica é construir por etapas uma resposta ótima combinando respostas já obtidas para partes menores
- Quando um algoritmo recursivo tem complexidade exponencial, a programação dinâmica pode entregar um algoritmo mais eficiente

Programação Dinâmica

- Em cada passo, sub-resultados são combinados obtendo-se respostas para partes maiores, até que se obtenha uma resposta para o problema original (similar com o comportamento de D&C)
- A decomposição é feita uma única vez e os casos menores são tratados antes dos maiores (normalmente os maiores derivam dos menores e suas soluções podem ser reutilizadas)
- Suas soluções são armazenadas para serem usadas quantas vezes for necessário

Programação Dinâmica



Programação Dinâmica

- É baseada no princípio da otimalidade
  - Em uma sequência ótima de escolhas ou de decisões, cada subsequência também deve ser ótima:
    - Exemplo: sucessão de caminhos mínimos de um ponto a outro (caminho de custo mínimo)
- Reduz drasticamente o número total de verificações pois evita aquelas que sabidamente não podem ser ótimas
  - A cada passo são eliminadas sub-soluções que certamente não compõem a solução ótima do problema

Programação Dinâmica - Exemplo

- Exemplo: Qual a melhor maneira de se fazer o produto entre n matrizes?
- O produto de uma matriz p × q por uma matriz q × r requer O(pqr) operações.
- Considerando o produto:

- $M = M_1[10, 20] \times M_2[20, 50] \times M_3[50, 1] \times M_4[1, 100]$:
  - O produto na ordem $M = M_1 \times (M_2 \times (M_3 \times M_4))$ requer 125.000 operações;
  - O produto na ordem $M = (M_1 \times (M_2 \times M_3)) \times M_4$ requer 2.200 operações.

Programação Dinâmica - Exemplo

- Resolver por tentativa e erro -> f(n) é um processo exponencial em n
  - $f(n) \geq 2^{(n-2)}$
- Usando programação dinâmica -> algoritmo com complexidade $O(n^3)$

Programação Dinâmica - Exemplo

- Seja m_{ij} o menor custo para calcular o produto $M_i \times M_{i+1} ... \times M_j$ para $1 \leq i \leq j \leq n$,
- Assim,

$$m_{ij} = \begin{cases} 0 & se\ i = j \\ Min_{i \leq k < i}(m_{ik} + m_{k+1,j} + b_{i-1} b_k b_j) & se\ j > i \end{cases}$$

  - onde:
    - o termo m_{ik} representa o custo mínimo para calcular $M' = M_i \times M_{i+1} \times ... \times M_k$,
    - o segundo termo $m_{k+i,j}$ representa o custo mínimo para calcular $M'' = M_{k+1} \times M_{k+2} \times ... \times M_j$,
    - o terceiro termo, $b_{i-1} b_k b_j$, representa o custo de multiplicar $M'[b_{i-1}, b_k]$ por $M''[b_k, b_j]$,
    - $b_{i-1} \times b_i$ são as dimensões da matriz M_i,
    - a equação acima diz que $m_{ij}, j > i$, representa o custo mínimo de todos os valores possíveis de k entre i e $j - 1$, da soma dos três termos.

Programação Dinâmica - Exemplo

- Seja m_{ij} o menor custo para calcular o produto $M_i \times M_{i+1} \ldots \times M_j$ para $1 \leq i \leq j \leq n$,
- Assim,

$$m_{ij} = \begin{cases} 0 \\ Min_{i \leq k \leq j} \end{cases}$$

Princípio da otimalidade: Em uma sequência ótima de escolhas ou de decisões, cada subsequência também deve ser ótima

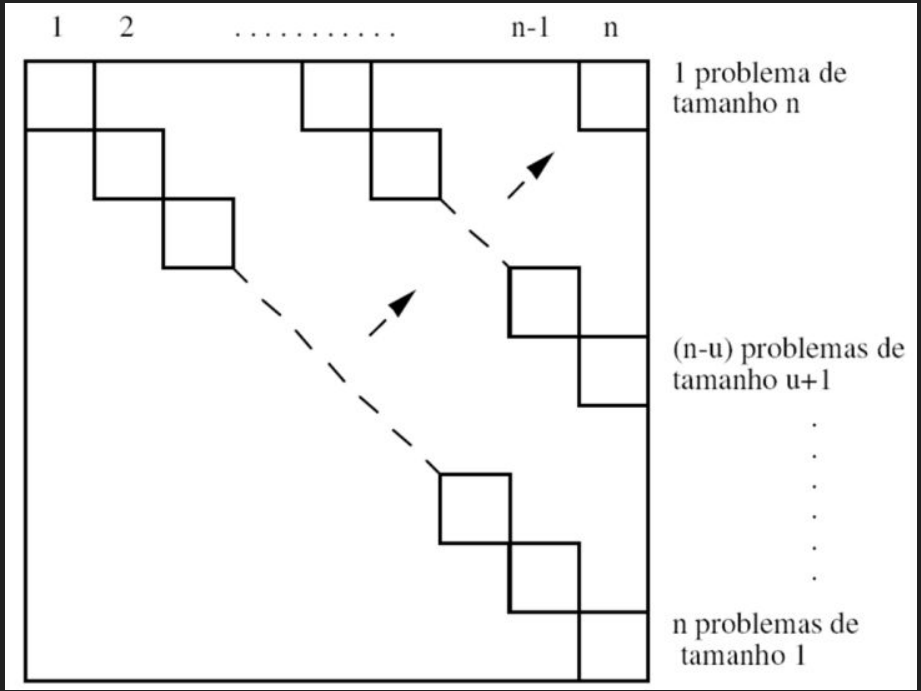
  - onde:
    - o termo m_{ik} rep[illegible] $M' = M_i \times M_{i+1} \times \ldots \times M_k$,
    - o segundo termo $m_{k+i,j}$ representa o custo mínimo para calcular $M'' = M_{k+1} \times M_{k+2} \times \ldots \times M_j$,
    - o terceiro termo, $b_{i-1} b_k b_j$, representa o custo de multiplicar $M'[b_{i-1}, b_k]$ por $M''[b_k, b_j]$,
    - $b_{i-1} \times b_i$ são as dimensões da matriz M_i,
    - a equação acima diz que $m_{ij}, j > i$, representa o custo mínimo de todos os valores possíveis de k entre i e $j-1$, da soma dos três termos.

Programação Dinâmica - Exemplo

- Os valores m_{ij} são calculados na ordem crescente das diferenças nos subscritos
  - o cálculo inicia com m_{ii} para todo i, depois m_{i,i+1} para todo i, depois m_{i,i+2}, ...
- O método é ascendente -> ao contrário dos métodos recursivos que são chamados descendentes

Programação Dinâmica - Exemplo

- Hierarquia de cálculos dos sub-problemas na PD



$M = M_1[10, 20] \times M_2[20, 50] \times M_3[50, 1] \times M_4[1, 100]$

$m_{11} = 0$	$m_{12} = 10.000$	$m_{13} = 1.200$	$m_{14} = 2.200$
	$m_{22} = 0$	$m_{23} = 1.000$	$m_{24} = 3.000$
		$m_{33} = 0$	$m_{34} = 5.000$
			$m_{44} = 0$

Algoritmos Gulosos

- São algoritmos que se baseiam na escolha que parece mais promissora no instante atual e nunca muda essa decisão, independentemente do que possa acontecer depois
  - São tipicamente usados para resolver problemas de otimização
- A cada iteração
  - seleciona um elemento conforme uma função gulosa e marca-o para não considerá-lo novamente nos próximos estágios
  - atualiza a entrada,
  - examina o elemento selecionado quanto a sua viabilidade
  - decide a sua participação ou não na solução

Algoritmos Gulosos

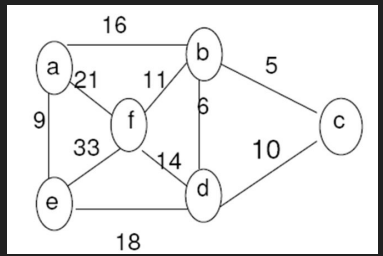
- Algoritmo guloso genérico
  - C: conjunto de candidatos
  - S: conjunto solução
  - S=∅
  - Enquanto (C != ∅) e (S não tem solução)
    - x = seleciona(C)
    - C = C − {x}
    - Se (S + {x} é viável) então S = S + {x}
  - Se (S tem solução) então retorna S
  - Senão não existe solução

Algoritmos Gulosos - Propriedades

- Para construir a solução ótima existe um conjunto ou lista de candidatos
  - São acumulados um conjunto de candidatos considerados viáveis para a solução
  - os demais candidatos são rejeitados para a solução
- Um conjunto de funções compõem um algoritmo guloso:
  - Uma função que verifica se um conjunto particular de candidatos produz uma solução (sem considerar otimalidade no momento)
  - Uma função que verifica se um conjunto de candidatos é viável (sem se preocupar com a otimalidade)
  - Uma função responsável pela seleção que indica a qualquer momento quais dos candidatos restantes é o mais promissor
  - Uma função objetivo que fornece o valor da solução encontrada
    - como por exemplo o comprimento do caminho construído (não aparece de forma explícita no algoritmo guloso)

Algoritmos Gulosos - Propriedades

- Não existe garantia de otimalidade
  - Quando funciona corretamente, a primeira solução encontrada é a ótima
- As funções de seleção e objetivo estão geralmente relacionadas, e se o objetivo é:
  - maximizar ⇒ provavelmente escolherá o candidato restante que proporcione o maior ganho individual
  - minimizar ⇒ então será escolhido o candidato restante de menor custo
- O algoritmo nunca muda de ideia
  - candidatos selecionados para o conjunto solução não são removidos
  - candidatos rejeitados não são mais revisitados
- Exemplo: Novamente o custo mínimo entre nós!



Algoritmos Aproximados

- Problemas considerados intratáveis ou difíceis são muito comuns
  - Problemas que somente possuem modelagens de solução usando algoritmos exponenciais
  - Para problemas difíceis é comum remover a exigência de que o algoritmo tenha sempre que obter a solução ótima global -> introduz a possibilidade de soluções ótimas locais de mais simples cálculo
- Exemplo: **problema do caixeiro viajante** cuja complexidade de tempo é O(n!)
  - Aproximado por transformação polinomial, ciclo de hilton (tentativa e erro ou branch-and-bound), heurísticas, etc

Algoritmos Aproximados

- **Heurística:** é uma regra geral ou uma estratégia que pode guiar a busca por uma solução ou uma resposta. Ao contrário de um algoritmo, uma heurística não garante um resultado correto ou ótimo, mas pode ajudá-lo a encontrar um bom o suficiente ou satisfatório em um tempo razoável
  - se baseia em conhecimento prévio sobre o problema para optar por algum caminho alternativo ao explorar TODAS as possibilidades
- **Algoritmo aproximado:** é um algoritmo que gera soluções aproximadas dentro de um limite para a razão entre a solução ótima e a produzida pelo algoritmo aproximado (comportamento monitorado sob o ponto de vista da qualidade dos resultados)

Para pensar

- **Como resolver o problema da mochila (qual estratégia ou paradigma empregar? Por quê?)?**



Dúvidas?

Obrigado!



Contato:

- la.souza@inf.ufes.br
- github.com/luisouza
- https://scholar.google.com.br/citations?user=3gMIEPYAAAAJ&hl=pt-BR
- https://www.researchgate.net/profile/Luis-Souza-Jr
- https://br.linkedin.com/in/luis-souza-22829368